University of East Anglia
COMPUTING SCIENCES

Advanced Web Development

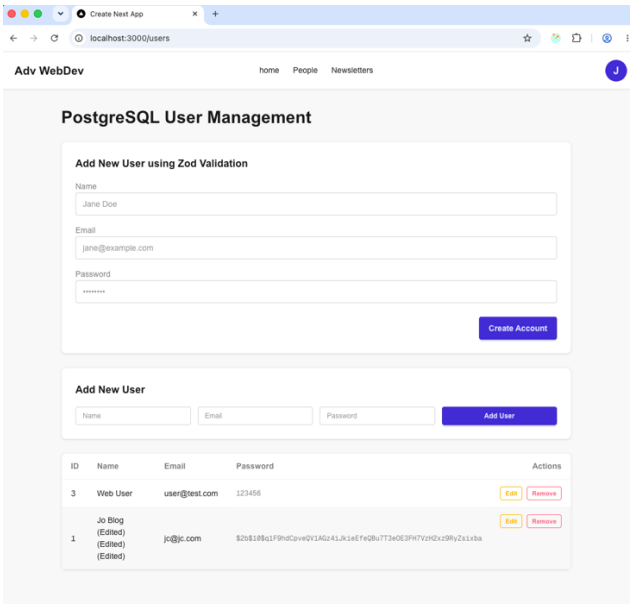
Exercise 4.1 Implementing Postgres DB with Prisma ORM and API end points

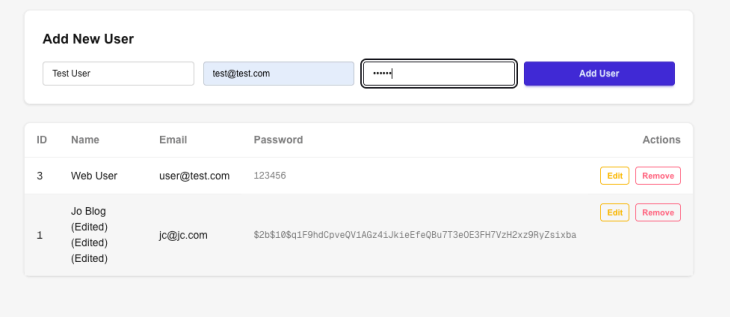
In the previous lab, we implemented social login using the Google Provider via Auth.js. With OAuth 2.0, the authentication heavy lifting is offloaded to a trusted Identity Provider. Your application never handles or stores a user's password but instead, it safely receives an identity token after Google verifies the user.

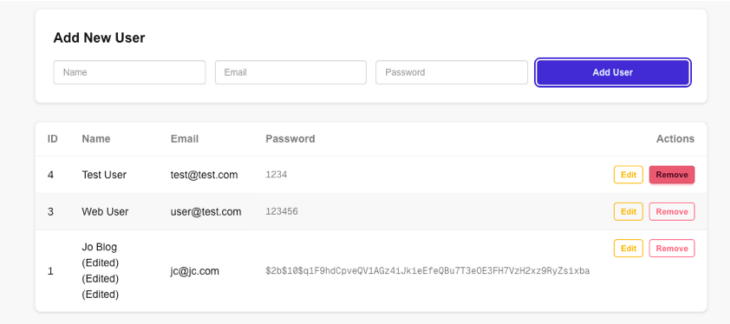
In this lab, we will configure a database and use Prisma ORM to perform migration. We will install the necessary dependency packages, code our database schema, and bridge Auth.js with our database using the Prisma Adapter. By the end of this lab, your application will be able to persist authenticated user profiles, manage active sessions, and store accounts seamlessly in a relational database.

To follow this lab, you must complete Excise 4" Implementing registration and login UIs with Authentication by Google Provider via Auth.js and custom hook" first.

Expected outputs:







Learning Objectives:

- Implement Postgres DB with Prisma ORM
- Write code to test the implementation
- Exploration of API end points

Tasks:

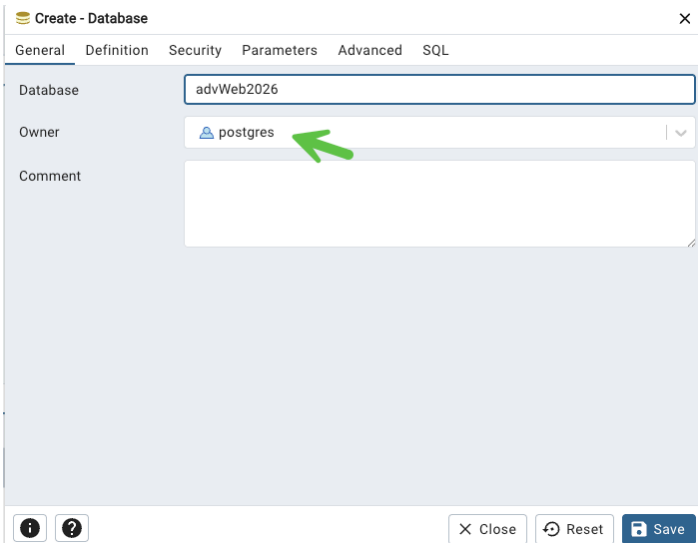
1. Create a new folder called 'lab4.1' and clone the code for lab4 inside this folder.
2. Open the folder on VSCode.
3. We will first create a database using PostgreSQL. You have been allocated a database on CMP PostgreSQL where you can use from a lab machine. You should have received an email from our IT team about the login details and database name. However, you are welcome to install it on your local machine if you are using your laptop (this is recommended). The following tasks are for those of you who use your own machine to complete the lab exercises, if you are using a lab machine, you can skip this!
    a. Use PGAdmin client and login to your local PostgresSQL server



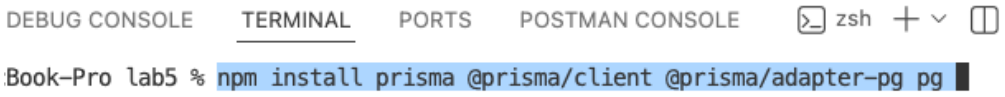
b. Create a new database called "advWeb2026"



c. Note down the database user and password



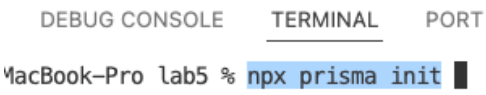
4. We will use Prisma 7 ORM to connect the database, and help with migration tasks. You can find more information on their website : https://www.prisma.io/docs/orm . First we will install the packages. On the terminal window, type:
`npm install prisma @prisma/client @prisma/adapter-pg pg`



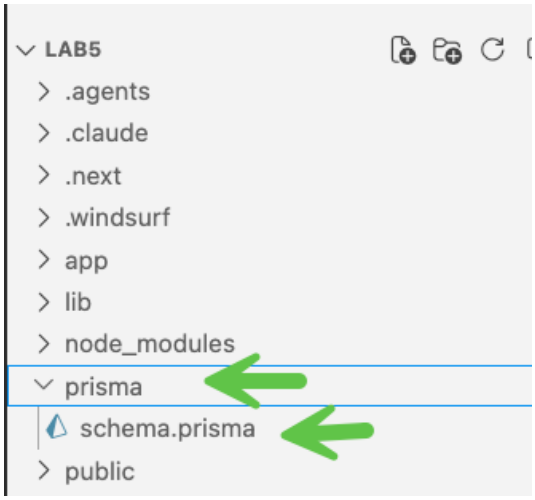
5. We also need to install the type:
`npm install -D @types/pg`



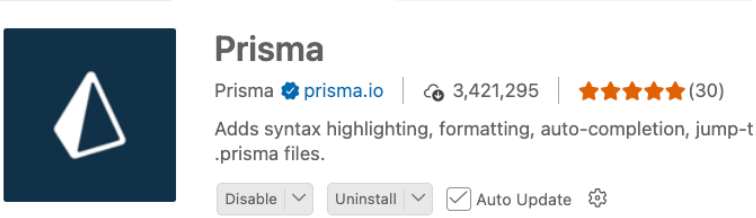
6. Now, type `npx prisma init`



7. Examine the project structure, you will see a folder named "prisma" being created, and inside this folder, there is a file named `schema.prisma`



8. Check your VSCode pluggin, I have Prisma installed you may wish to do so if you have not already done so:



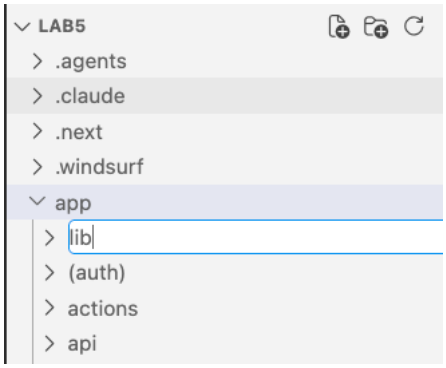
9. Open the `schema.prisma` file, you will see the `client` and `db` have already been generated:

```
prisma > schema.prisma > db
1   // This is your Prisma schema file,
2   // learn more about it in the docs: https
3
4   // Get a free hosted Postgres database in
5
    Generate
6   generator client {
7     provider = "prisma-client"
8     output   = "../lib/generated/prisma"
9   }
10
11  datasource db {
12    provider = "postgresql"
13  }
14
```

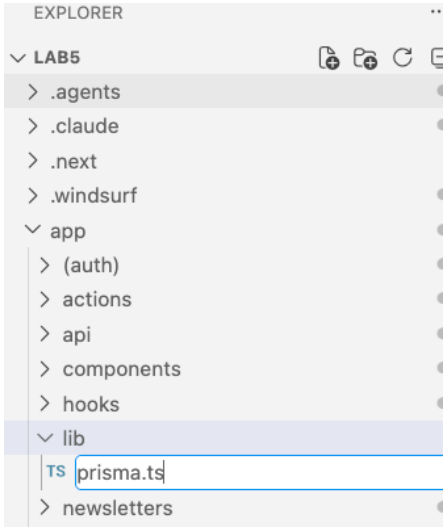
10. Now we want to code our user model, do the following:

```
15  model User {
16    id        Int      @id @default(autoincrement())
17    email     String   @unique
18    password  String
19    name      String
20    createdAt DateTime @default(now()) @map("created_at")
21    updatedAt DateTime @updatedAt @map("updated_at")
22
23
24    @@map("users")
25  }
26
27
```

11. Create a database connection:
    a. Inside the 'app' folder, create a new folder called 'lib'



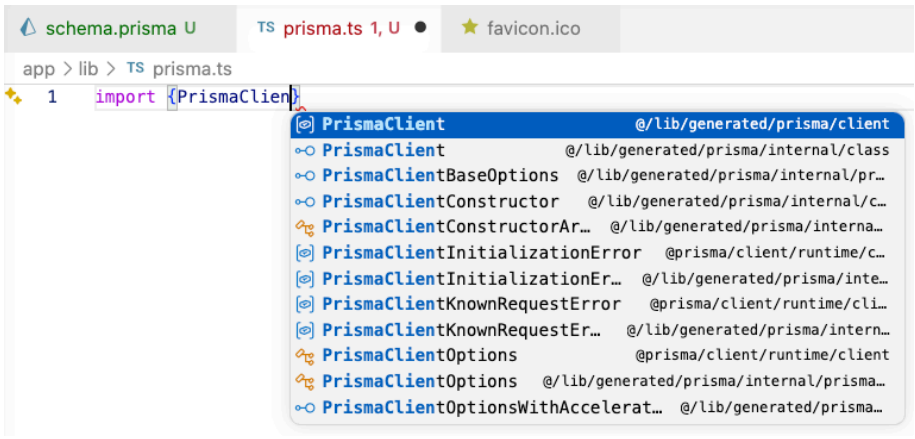
    b. Inside this folder create a new file called '`prisma.ts`'

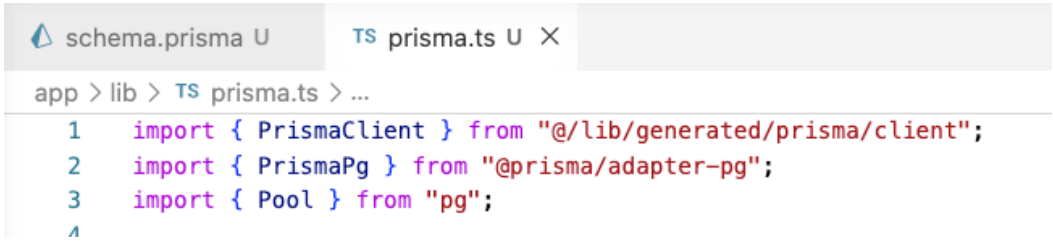


    c. On the terminal window type:



d. Open this `prisma.ts` file, and import the following:





e. If you run into trouble of importing `PrismaClient`, then check your `schema.prisma`, my configurations are shown below. Make sure that you type "`npx prisma generate`" first

```
Generate
generator client {
  provider = "prisma-client"
  output   = "../lib/generated/prisma"
}

datasource db {
  provider = "postgresql"
}
```

f. Paste below code to `prisma.ts` to create Prisma Client for connecting to database. Here we are implementing Postgres connection poll to allow HTTP requests using few database connection.

```
const globalForPrisma = globalThis as unknown as {
  prisma: PrismaClient | undefined;
};

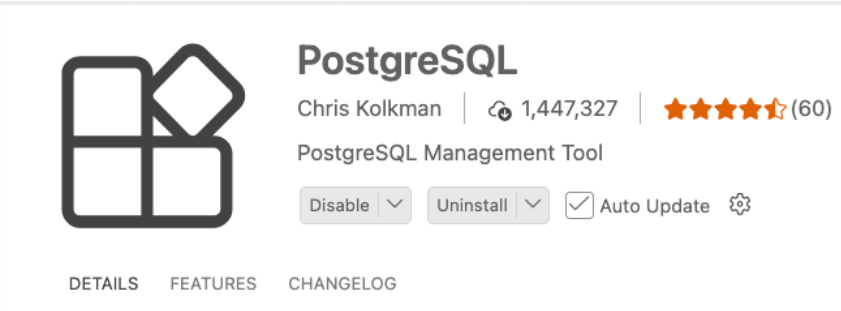
function createPrismaClient() {
  const pool = new Pool({ connectionString:
process.env.DATABASE_URL });
  const adapter = new PrismaPg(pool);
  return new PrismaClient({ adapter });
}

export const prisma = globalForPrisma.prisma ?? createPrismaClient();

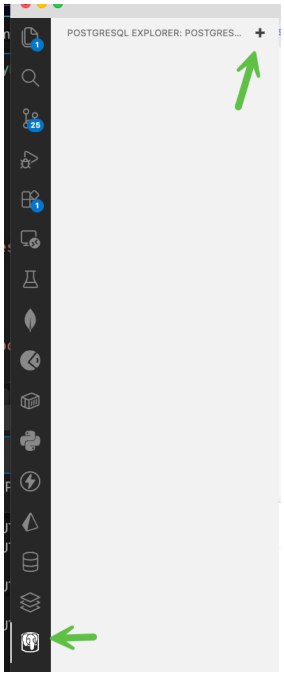
if (process.env.NODE_ENV !== "production") {
  globalForPrisma.prisma = prisma;
}
```



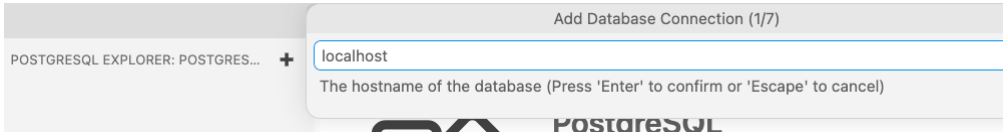
g. If you have not got a Postgres client installed on your machine, install PostgresSQL plugin on your VSCode:



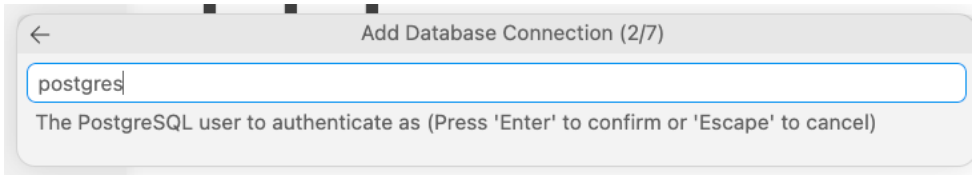
h. Click the database explorer and create a new connection



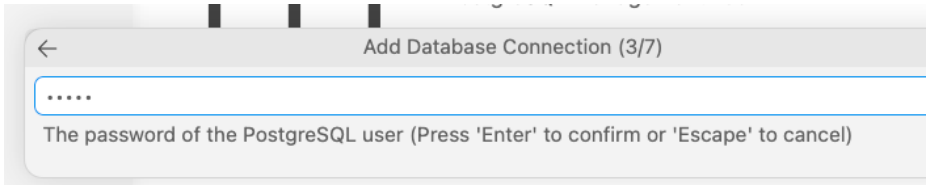
i. Enter localhost



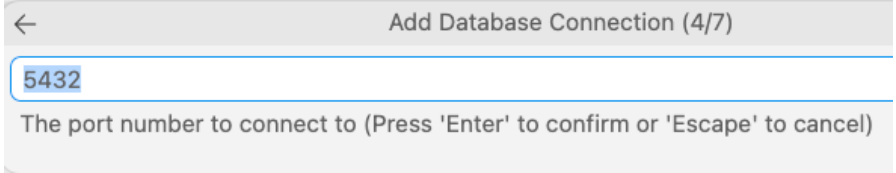
j. Enter the user



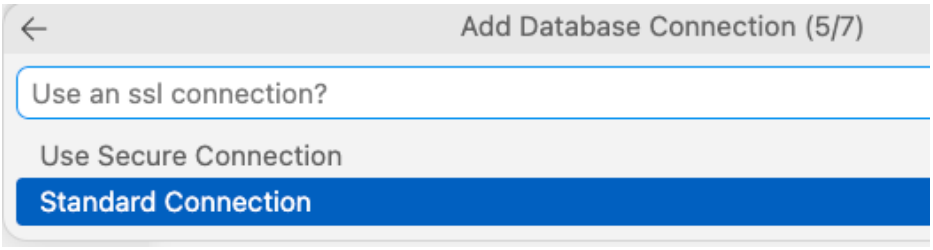
k. Enter the password



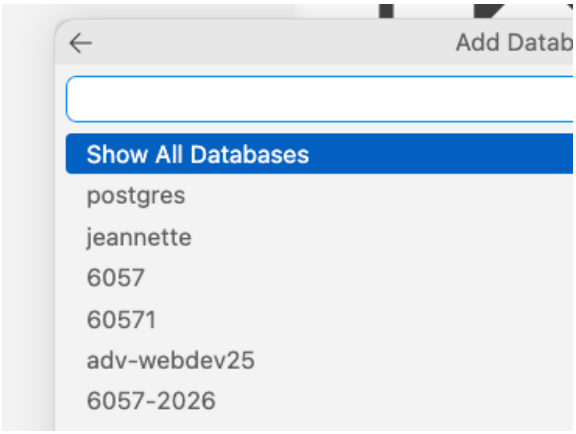
l. The port is 5432



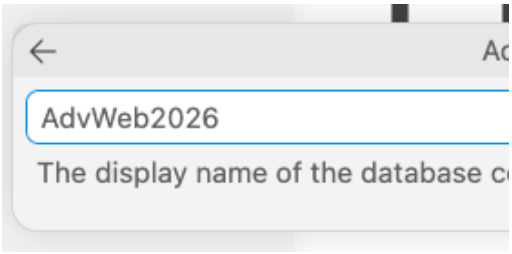
m. Select Standard Connection



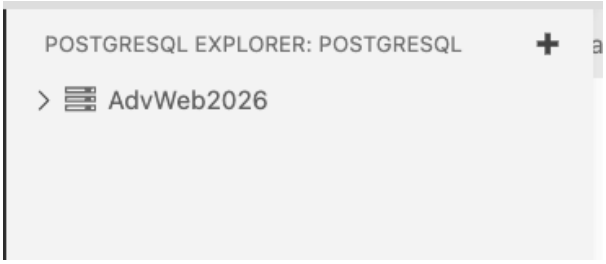
n. Select your database from the dropdown list, my database is "AdvWeb2026'



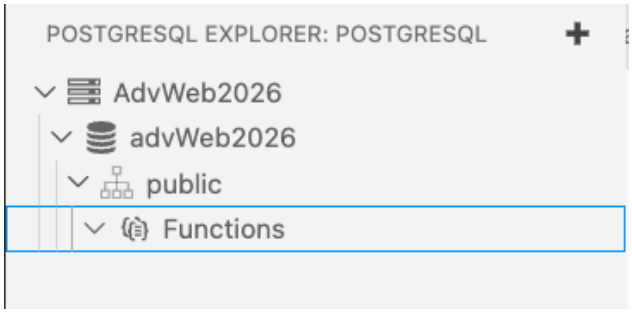
o. Give this connection a name:



p. You will see the connection listed on the panel



q. Examine the database, currently it is empty:



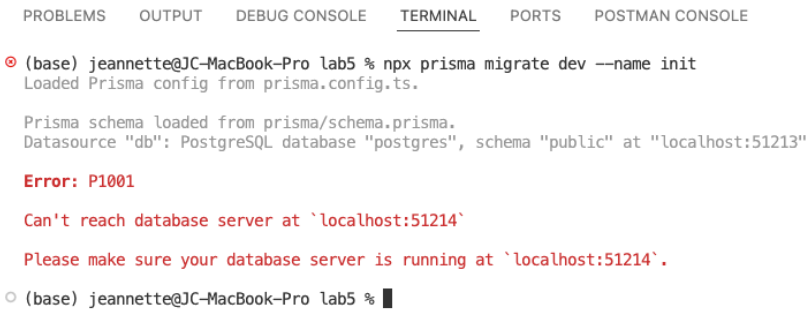
r. Now we start our migration. Bring up a new terminal window:



s. Type "`npx prisma migrate dev --name init`"



t. If you see this error –



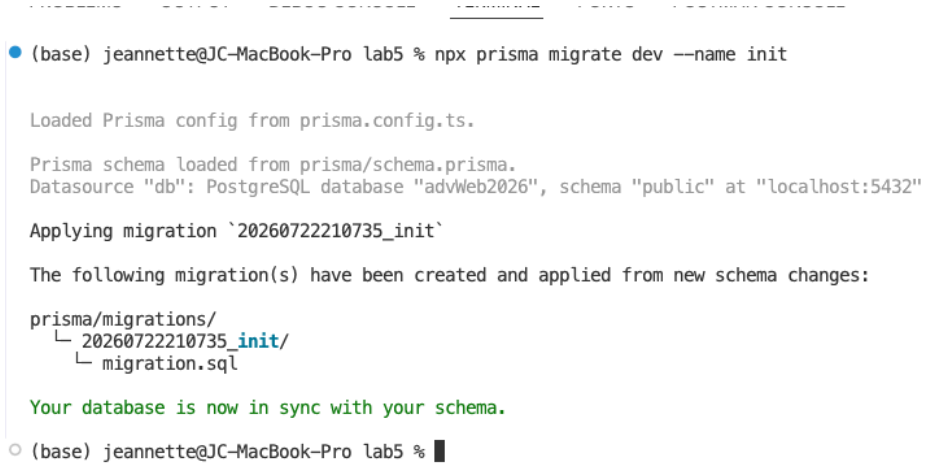
u. Open `.env` file , create or modify the database connection string with the following, replace the 'postgres' with your database user, '12345' with your password, and 'advWeb2026' with your own database configurations.



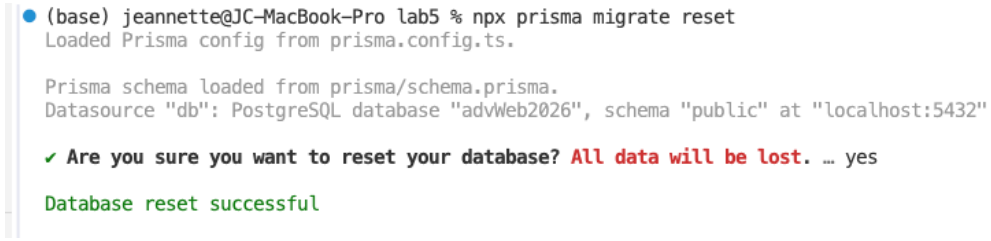
v. Now try again, type `npx prisma migrate dev --name init`



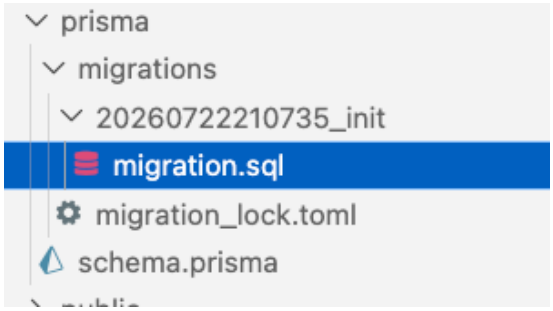
w. You will see the output similar to below:



x. To reset the database, simple type " `npx prisma migrate reset` "– and type Y when prompt:



y. Examine the 'prisma' folder, you will the migration `sql` is generated there



prisma > migrations > 20260722210735_init > migration.sql

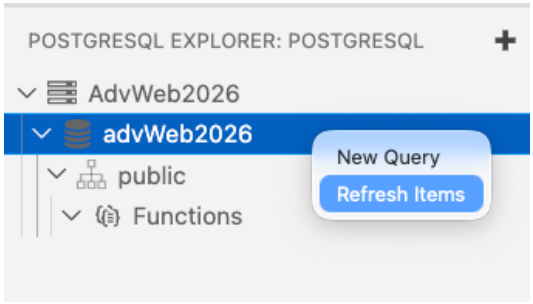
```sql
-- CreateTable
CREATE TABLE "users" (
    "id" SERIAL NOT NULL,
    "email" TEXT NOT NULL,
    "password" TEXT NOT NULL,
    "name" TEXT NOT NULL,
    "created_at" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "updated_at" TIMESTAMP(3) NOT NULL,

    CONSTRAINT "users_pkey" PRIMARY KEY ("id")
);

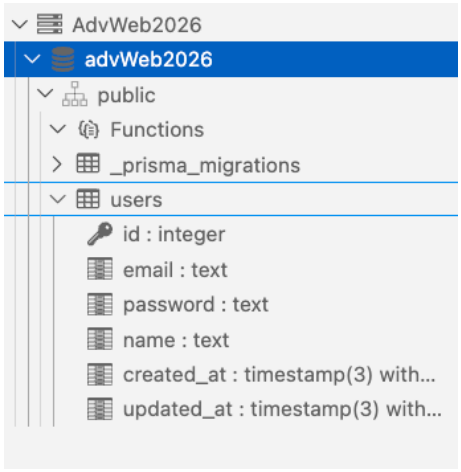
-- CreateIndex
CREATE UNIQUE INDEX "users_email_key" ON "users"("email");

```

z. Go to Postgres explorer or your Postgres client and refresh the items

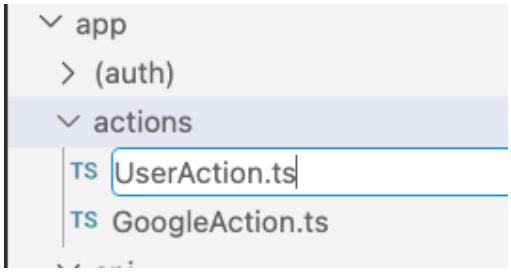


aa. Notice that the table is now generated



We have now configured our database and able to connect. We will now code some functions to test it!

12. Create some helper functions:



13. Open this file:
    a. Make it a server component and import the following:

app > actions > TS UserAction.ts > removeUser

```ts
"use server";

import { prisma } from "../lib/prisma";
import { revalidatePath } from "next/cache";

```

    b. Code an add user function as follow:

```ts
// --- 1. ADD USER ---
export async function addUser(data: {
  name: string;
  email: string;
  pass: string;
}) {
  try {
    const newUser = await prisma.user.create({
      data: {
        name: data.name,
        email: data.email,
        password: data.pass,
      },
    });

    // Refresh the page cache to display the new user immediately
    revalidatePath("/users");
    return { success: true, user: newUser };
  } catch (error) {
    console.error("Failed to add user:", error);
    return {
      success: false,
      error: "Failed to create user. Email may already exist.",
    };
  }
}
```

c. Code an edit user function as follow:

```
32
33  // --- 2. EDIT USER ---
34  export async function editUser(
35    id: number,
36    data: { name?: string; email?: string; pass?: string },
37  ) {
38    try {
39      const updatedUser = await prisma.user.update({
40        where: { id },
41        data: {
42          ...(data.name && { name: data.name }),
43          ...(data.email && { email: data.email }),
44          ...(data.pass && { password: data.pass }),
45        },
46      });
47
48      revalidatePath("/users");
49      return { success: true, user: updatedUser };
50    } catch (error) {
51      console.error("Failed to update user:", error);
52      return { success: false, error: "Failed to update user." };
53    }
54  }
55
```

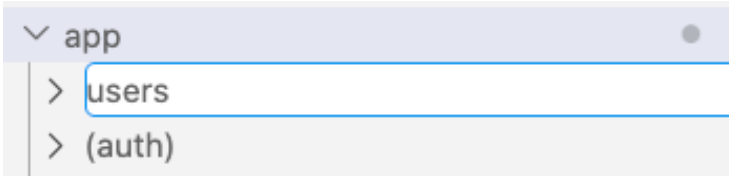
d. Code a remove user function as follow:

```
56  // --- 3. REMOVE USER ---
57  export async function removeUser(id: number) {
58    try {
59      const deletedUser = await prisma.user.delete({
60        where: { id },
61      });
62
63      revalidatePath("/users");
64      return { success: true, user: deletedUser };
65    } catch (error) {
66      console.error("Failed to remove user:", error);
67      return { success: false, error: "Failed to remove user." };
68    }
69  }
70
```

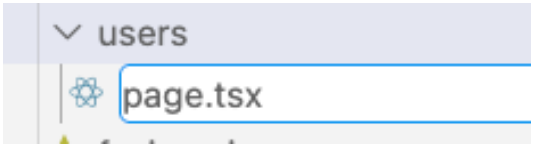
e. Code a get all users function as follow:

```
70
71  // --- 4. GET ALL USERS (Helper to render the list) ---
72  export async function getUsers() {
73    return await prisma.user.findMany({
74      orderBy: { id: "desc" },
75    });
76  }
77
```

14. Now create `users` path:



15. And the `page.tsx`



16. Open this `page.tsx` and do the following:
    a. Import the following:

```
app > users > page.tsx > UsersPage
1  import { JSX } from "react";
2  import { getUsers, addUser, editUser, removeUser } from "../actions/UserAction";
3
```

    b. Code the users page (it's async function), first get the list of users

```
3
4  export default async function UsersPage(): Promise<JSX.Element> {
5    const users = await getUsers();
6
```

c. Code an add user <form> using form action to add the user:

```
{/* Form using Server Action */}
<form
  action={async (formData: FormData) => {
    "use server";
    const name = formData.get("name") as string;
    const email = formData.get("email") as string;
    const pass = formData.get("pass") as string;

    if (name && email && pass) {
      await addUser({ name, email, pass });
    }
  }}
  className="grid grid-cols-1 md:grid-cols-4 gap-3 items-end mt-2"
>
  <input
    type="text"
    name="name"
    placeholder="Name"
    className="input input-bordered input-sm w-full"
    required
  />
  <input
    type="email"
    name="email"
    placeholder="Email"
    className="input input-bordered input-sm w-full"
    required
  />
  <input
    type="password"
    name="pass"
    placeholder="Password"
    className="input input-bordered input-sm w-full"
    required
  />
  <button type="submit" className="btn btn-primary btn-sm">
    Add User
  </button>
</form>
</div>
```

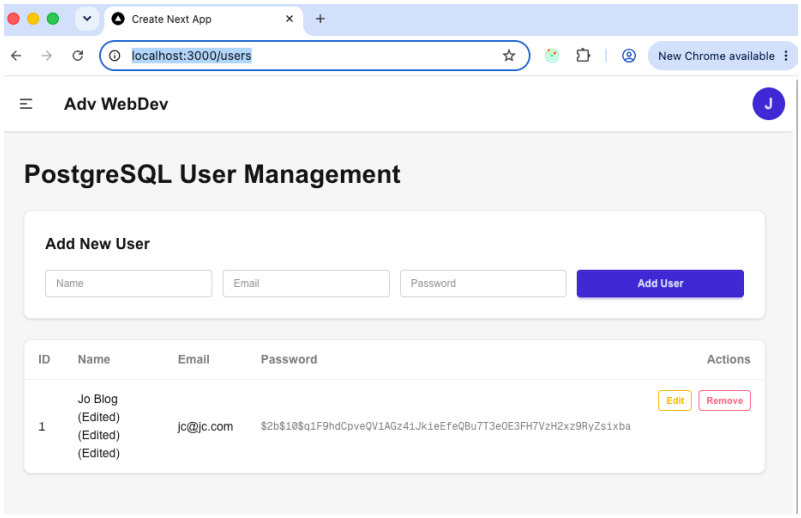
d. Code a `<table>` and render the users found in the database, one on each row, and provide 2 buttons – edit and remove (I render all fields, including password for testing purposes!)

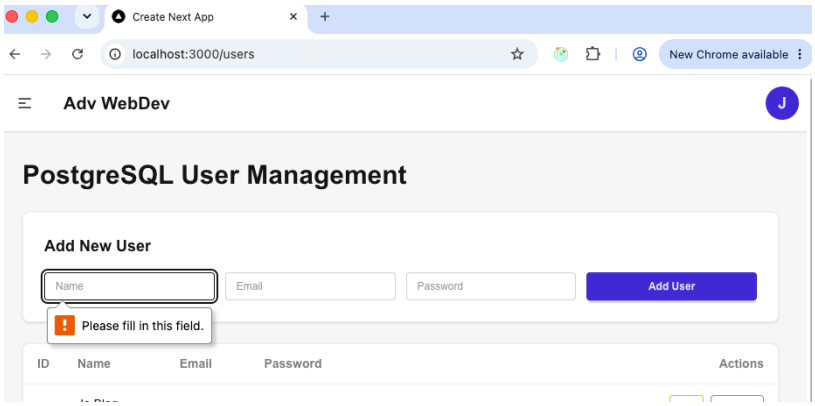
```
<table className="table table-zebra w-full">
  <thead>
    <tr>
      <th>ID</th>
      <th>Name</th>
      <th>Email</th>
      <th>Password</th>
      <th className="text-right">Actions</th>
    </tr>
  </thead>
  <tbody>
    {users.map((user) => (
      <tr key={user.id}>
        <td className="font-mono">{user.id}</td>
        <td>{user.name}</td>
        <td>{user.email}</td>
        <td className="font-mono text-xs opacity-60">
          {user.password}
        </td>
        <td className="text-right flex justify-end gap-2">
          {/* EDIT BUTTON (QUICK NAME UPDATE) */}
          <form
            action={async () => {
              "use server";
              await editUser(user.id, {
                name: `${user.name} (Edited)`,
              });
            }}
          >
            <button
              type="submit"
              className="btn btn-xs btn-outline btn-warning"
            >
              Edit
            </button>
          </form>

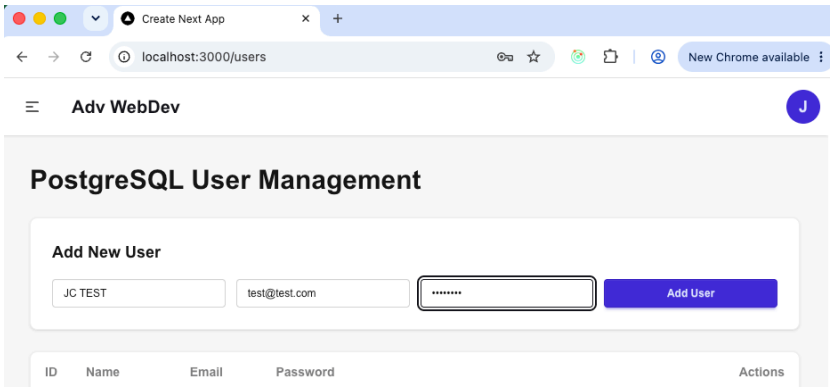
          {/* DELETE BUTTON */}
          <form
            action={async () => {
              "use server";
              await removeUser(user.id);
            }}
          >
            <button
              type="submit"
              className="btn btn-xs btn-outline btn-error"
            >
              Remove
            </button>
          </form>
        </td>
      </tr>
    ))}

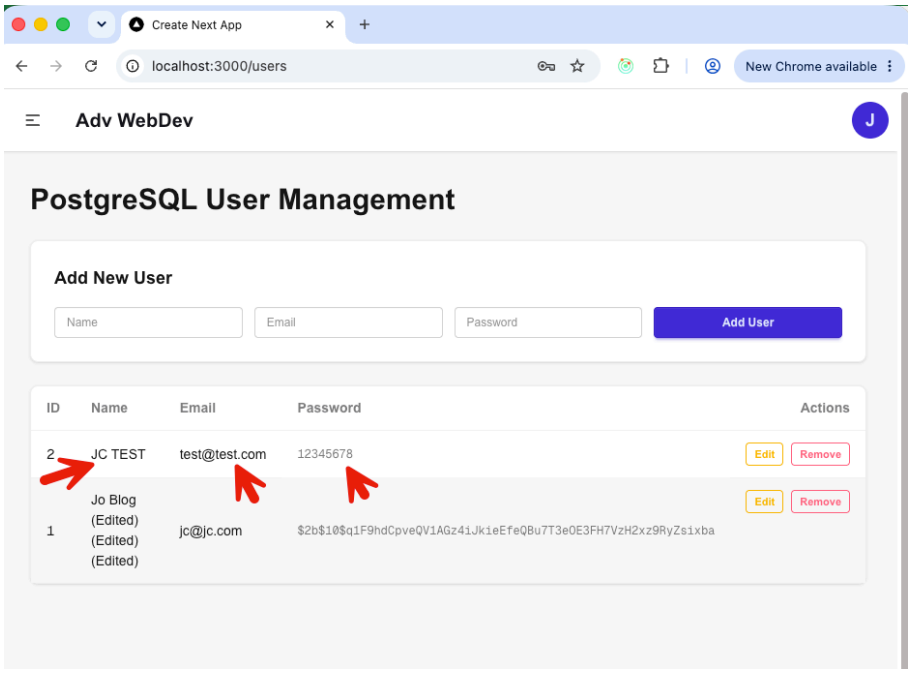
    {users.length === 0 && (
      <tr>
        <td
          colSpan={5}
          className="text-center py-4 text-base-content/60"
        >
          No users found in database. Add one above!
        </td>
      </tr>
    )}
  </tbody>
</table>
```

e. Navigate to http://localhost:3000/users and test it!





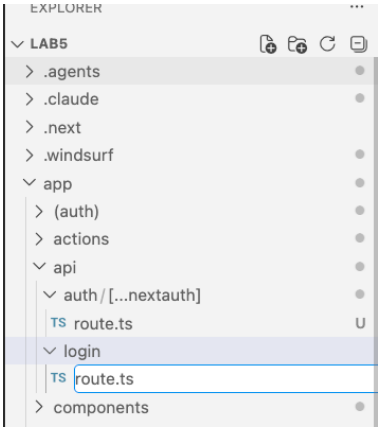




As you can see, the password is captured in plain text! We will implement bcrypt encryption next week.

17. We will examine the end points next. Inside the `app/api` folder, create a new folder called '`login`'

18. Inside this new folder, create a new file called '`route.ts`'



19. Open this `route.ts` file:
    a. Import the request and response from next/server

```
app > api > login > TS route.ts > ...
1   import { NextRequest, NextResponse } from "next/server";
2
```

b. Code a POST route (i.e. function):

```
3   export async function POST(request: NextRequest) {
4
5   }
```

c. Inside the POST route:

i. first, retrieve the request json

```
4
5     const { email, password } = await request.json();
```

ii. code a dummy check if the login credentials meet with predefined values, if not, return a message with status 401

```
11    if (email !== "admin" || password !== "admin") {
12      return NextResponse.json({ error: "Invalid Credentials" }, { status: 401 });
13    }
```

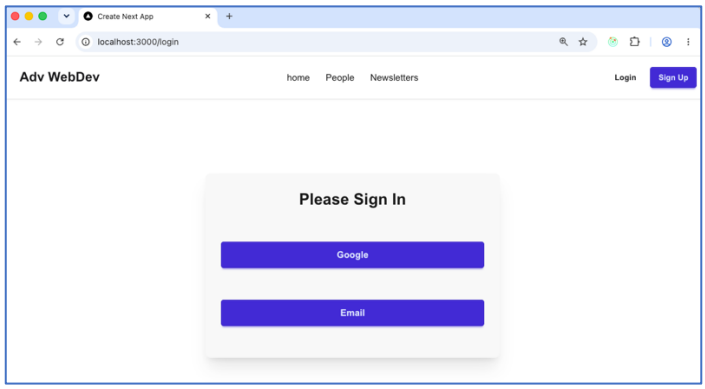
d. Now code the positive scenario for the login credentials meet the predefined values

```
17
18    const response = NextResponse.json({
19      message: "Login Successful",
20      user: { id: "1", name: "admin" },
21    });
```

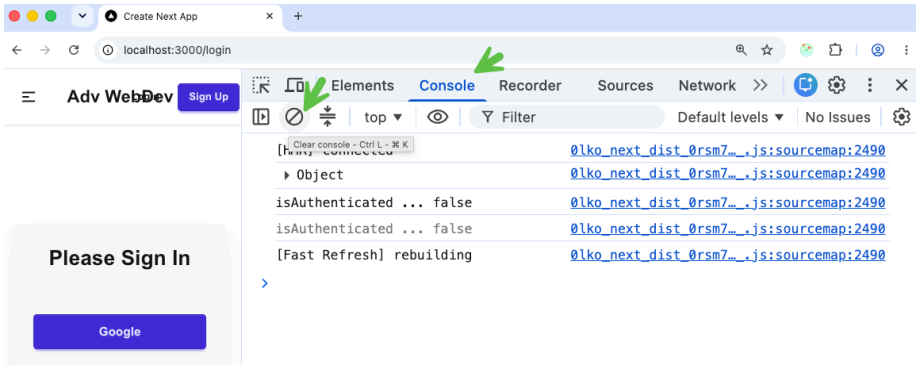
e. Finally return the response:

```
26
27    return response;
```

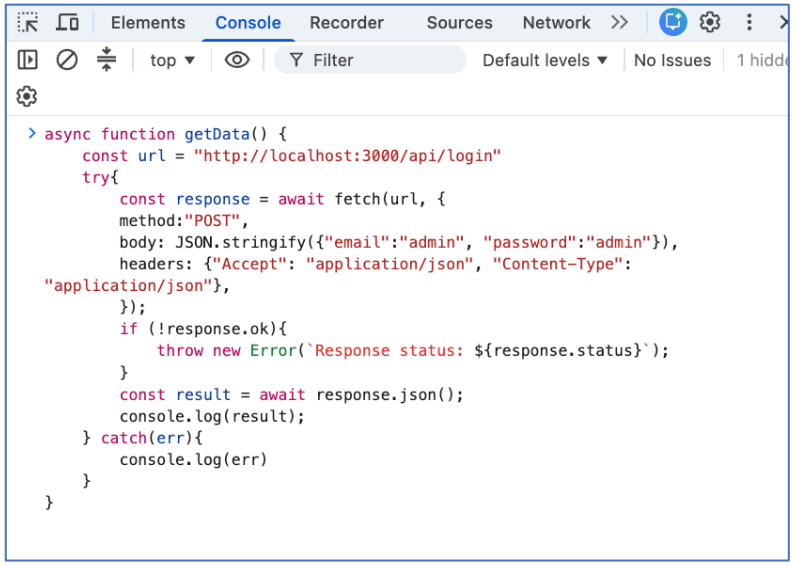
f. Bring up your development server, and navigate to localhost:3000 on Chrome. You should see the page looks like below:



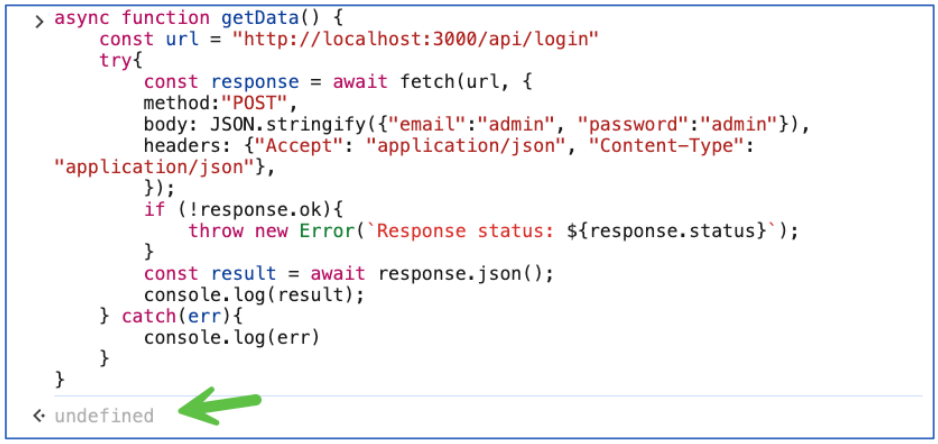
g. Go to console tab, and clear the console:



h. Then type below function (request with the correct credentials) on the console, and hit enter



i. You will see this 'undefined' – why?



j. Now execute the function:

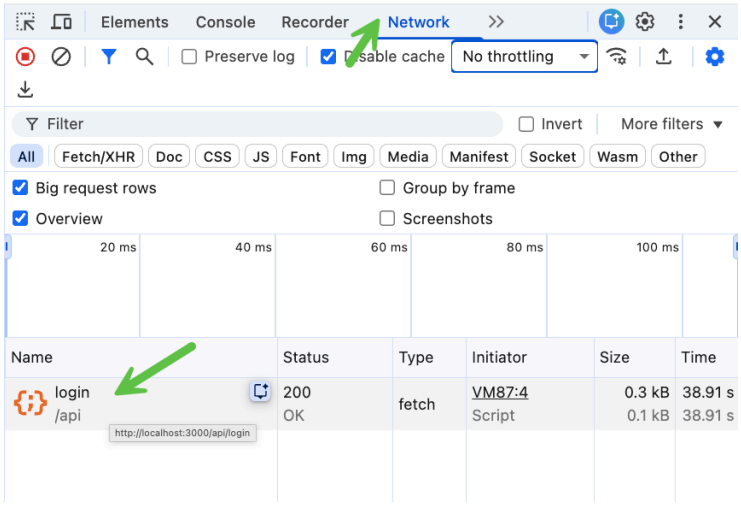
```
> async function getData() {
      const url = "http://localhost:3000/api/login"
      try{
          const response = await fetch(url, {
          method:"POST",
          body: JSON.stringify({"email":"admin", "password":"admin"}),
          headers: {"Accept": "application/json", "Content-Type":
  "application/json"},
          });
          if (!response.ok){
              throw new Error(`Response status: ${response.status}`);
          }
          const result = await response.json();
          console.log(result);
      } catch(err){
          console.log(err)
      }
  }
<- undefined
> getData()
```

k. Observe the results returned:

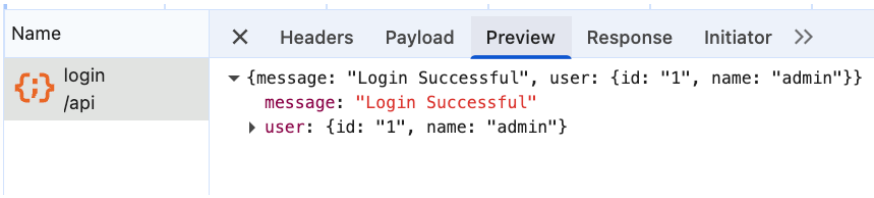
```
> async function getData() {
      const url = "http://localhost:3000/api/login"
      try{
          const response = await fetch(url, {
          method:"POST",
          body: JSON.stringify({"email":"admin", "password":"admin"}),
          headers: {"Accept": "application/json", "Content-Type":
  "application/json"},
          });
          if (!response.ok){
              throw new Error(`Response status: ${response.status}`);
          }
          const result = await response.json();
          console.log(result);
      } catch(err){
          console.log(err)
      }
  }
<- undefined
> getData();
<- ▸ Promise {<pending>}
```

```
  }
<- undefined
> getData();
<- ▾ Promise {<pending>} i
    ▾ [[Prototype]]: Promise
      ▸ catch: ƒ catch()
      ▸ constructor: ƒ Promise()
      ▸ finally: ƒ finally()
      ▸ then: ƒ then()
        Symbol(Symbol.toStringTag): "Promise"
      ▸ [[Prototype]]: Object
        [[PromiseState]]: "pending"
        [[PromiseResult]]: undefined
  ▾ {message: 'Login Successful', user: {…}} i                VM87:13
      message: "Login Successful"
    ▸ user: {id: '1', name: 'admin'}
    ▸ [[Prototype]]: Object
```

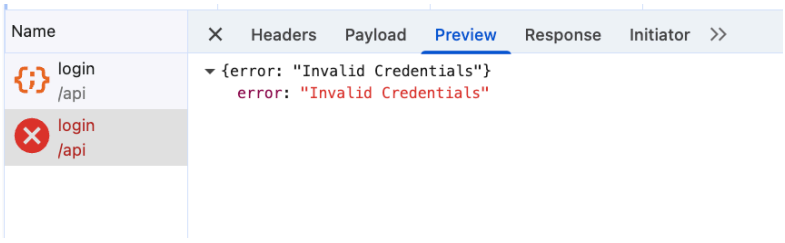
l. Now go to the 'Network' tab and select the "Login" api:



m. Examine the Preview, Headers, and Payload.



n. Now repeat the same function but with an incorrect password, and examine the outputs! You should see the error in Preview as follow:



Well done for completing this lab. We will implement Auth.js custom credentials implementation next week!

Push your code to CMP Stugit and name your repo as "<your-username>lab4.1"